МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Языки программирования

Шаблоны и исключения

Руководитель _____ А.В. Проскурин
подпись, дата инициалы, фамилия

Обучающийся БПИ24-02, 24151430 _____ В.Т. Гордов
номер группы, зачетной книжки подпись, дата инициалы, фамилия

Красноярск 2025 г.

ЦЕЛЬ РАБОТЫ

Получение практических навыков разработки и отладки программ с использованием шаблонов классов и механизма обработки исключений.

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Ознакомиться с общей постановкой задачи.
2. Ознакомится с вариантом задания – соответствует вашему номеру в списке группы
3. Разработать классы согласно варианту задания.
4. Реализовать механизм обработки исключений для обеспечения надежности работы программы.
5. Написать и отладить программу на подготовленных наборах тестовых данных.
6. Подготовить отчет по лабораторной работе. Отчет должен включать в себя:
    - титульный лист;
    - цель лабораторной работы;
    - постановку задачи;
    - текст программы с комментариями;
    - демонстрацию работы программы (Снимки экрана при выполнении действий программы с описанием).
    - краткие ответы на контрольные вопросы;
    - выводы по лабораторной работе.
7. Защитить лабораторную работу перед преподавателем.

ПОСТАНОВКА ЗАДАЧИ

Необходимо разработать программу, в которой будет реализован шаблонный класс, выполняющий обработку массива данных согласно варианту задания. **Особое внимание уделить обработке исключительных ситуаций.**

Общее задание для всех вариантов (Обязательная часть, связанная с исключениями):

1. **Конструкторы и деструктор**: Реализовать конструктор по умолчанию, конструктор копирования, конструктор перемещения (move constructor), оператор копирующего и перемещающего присваивания. Обеспечить строгую гарантию исключений (strong exception safety) для всех методов, изменяющих состояние объекта.

2. **Безопасность методов**: Все методы, работающие с динамической памятью или выполняющие критичные операции, должны обеспечивать как минимум базовую гарантию исключений (no resource leaks).

3. **Проверка входных данных**: Методы, получающие параметры извне (например, индексы), должны проверять их корректность (выход за границы массива, нулевые указатели и т.д.) и генерировать соответствующие исключения (например, std::out_of_range, std::invalid_argument).

4. **Обработка исключений в main**: Продемонстрировать обработку исключений в функции main() с помощью try-catch блоков, обеспечивая корректное завершение программы или восстановление после ошибок.

Дополнительные задания (в зависимости от оценки):

Оценка «Удовлетворительно» (Базовый уровень)

Что должен сделать студент:

1. Выполнить обязательную часть.
2. Написать методы ввода/вывода данных, принимающие потоки в качестве параметров.
3. Реализовать один метод из варианта..

Оценка «Хорошо» (Продвинутый уровень)

Что должен сделать студент

1. **Доработать программу с оценки «Удовлетворительно».**
2. Перегрузить операции ввода/вывода (>>, <<), вызвав в них методы ввода/вывода.
3. Выполнить второй метод из варианта.

Оценка «Отлично» (Углубленный уровень)

Что должен сделать студент

1. **Доработать программу с оценки «Хорошо».**
2. Реализовать собственный класс исключений. Создать хотя бы один собственный класс исключения, унаследованный от std::exception, для обработки специфичных для вашей предметной области ошибок (например, FigureValidationError, MatrixDimensionError).

3. Реализовать дополнительные методы (например, GenerateRandomValues(), модификация элемента, перегрузка операторов сравнения).

Вариант №5.

Напишите программу, в которой создайте шаблонный класс **TMatrix**, реализующий динамический двумерный массив и функции-методы:

- подсчета элементов массива (с отображением на экран), которые являются положительными и меньше среднеарифметического значения всех элементов своей строки;
- расчёта суммы отрицательных элементов массива.

Протестируйте вашу программу для работы с массивами различных типов: **float**, класса **дробное число** (которое представлено двумя компонентами). Этот класс необходимо создать.

ХОД РАБОТЫ

Файл **TMatrix.h**:

```cpp
#ifndef TMATRIX_H
#define TMATRIX_H

#include <iostream>
#include <fstream>
#include <vector>
#include <random>
#include <limits>
#include <algorithm>
#include <stdexcept>
#include <type_traits>
using namespace std;

/**
 * @brief Шаблонный класс, реализующий динамический двумерный массив
 * и предоставляющий методы для работы с ним.
 *
 * @tparam T Тип элементов матрицы:
 * float (вещественное число)
 * Fraction (дробь вида a/b)
 */
You, 3 минуты назад | 2 authors (PlayerUnknown14 and one other)
template<typename T>
class TMatrix {
private:
    // Представление матрицы как вектора векторов (двумерный массив)
    vector<vector<T>> data;
    // Количество строк матрицы
    int rows;
    // Количество столбцов матрицы
    int cols;

public:
    // Конструктор по умолчанию (пустая матрица 0x0)
    TMatrix();

    // Конструктор с размерами (r x c)
    TMatrix(int r, int c);

    // Конструктор копирования
    TMatrix(const TMatrix& other);

    // Конструктор перемещения
    TMatrix(TMatrix&& other) noexcept;

    // Деструктор
    ~TMatrix();

    // Оператор присваивания
    TMatrix& operator=(const TMatrix& other);

    // Оператор перемещающего присваивания
    TMatrix& operator=(TMatrix&& other) noexcept;

    // Оператор сравнения
    bool operator==(const TMatrix& other) const;

    // Методы ввода/вывода данных матрицы
    void input(istream& is = cin);
    void output(ostream& os = cout) const;

    // Перегрузки операторов ввода/вывода (обёртка соответствующих методов)
    friend istream& operator>>(istream& is, TMatrix& m) {
        if (is.fail()) {
            throw runtime_error("TMatrix Output: Failed to write dimensions to stream.");
        }
        m.input();
        return is;
    }
```

```cpp
    friend ostream& operator<<(ostream& os, const TMatrix& m) {
        if (os.fail()) {
            throw runtime_error("TMatrix Output: Failed to write dimensions to stream.");
        }
        m.output();
        return os;
    }

    // Подсчёт элементов > 0 и < среднего по строке
    int countPositiveBelowRowMean() const;
    // Подсчёт суммы отрицательных элементов
    T sumOfNegativeElements() const;

    // Заполнение матрицы случайными значениями
    void GenerateRandomValues(T min_val, T max_val);

    // Модификация элемента матрицы (ввод с консоли)
    void modifyElement(int row, int col, istream& is);

    // Модификация элемента матрицы (случайное значение)
    void SetRandomValue(int row, int col, T min_val, T max_val);

    // Загрузка из файла
    void loadFromFile(const string& filename);

    // Сохранение в файл
    void saveToFile(const string& filename) const;

    // Проверка границы матрицы
    const T& at(int row, int col) const;
};

// --- РЕАЛИЗАЦИЯ МЕТОДОВ ШАБЛОНА ---

template<typename T>
TMatrix<T>::TMatrix() : rows(0), cols(0) {}

/**
 * @brief Конструктор с параметрами.
 * Создаёт матрицу размером `r x c`, заполняя элементы значением по умолчанию `T()`.
 *
 * @param r Количество строк.
 * @param c Количество столбцов.
 * @throws `invalid_argument` если r < 0 или c < 0.
 * @throws `length_error` если `r * c` превышает максимальный размер `vector`.
 */
template<typename T>
TMatrix<T>::TMatrix(int r, int c) : rows(r), cols(c) {
    if (r < 0 || c < 0) {
        throw invalid_argument("TMatrix Constructor: Dimensions cannot be negative!");
    }
    // Проверка на переполнение при умножении (опционально, но безопасно)
    if (r > 0 && c > numeric_limits<int>::max() / r) {
        throw length_error("TMatrix Constructor: Dimensions are too large, would cause overflow!");
    }
    try {
        data.resize(rows, vector<T>(cols));
    }
    catch (const bad_alloc& e) { // Обработка исключения при выделении памяти
        // Возвращаем объект в корректное состояние (пустая матрица)
        rows = 0;
        cols = 0;
        throw;
    }
}
```

```cpp
/**
 * @brief Конструктор копирования.
 * Создаёт новую матрицу, являющуюся копией переданной.
 *
 * @param other Матрица, из которой производится копирование.
 * @throws `bad_alloc` при нехватке памяти для копирования.
 */
template<typename T>
TMatrix<T>::TMatrix(const TMatrix& other) : rows(other.rows), cols(other.cols) {
    try {
        data = other.data;
    }
    catch (const bad_alloc& e) {
        // Возвращаем объект в корректное состояние (пустая матрица)
        rows = 0;
        cols = 0;
        data.clear();
        throw;
    }
}

/**
 * @brief Конструктор перемещающего копирования.
 * Создаёт новую матрицу, являющуюся копией переданной.
 *
 * @param other Матрица, из которой производится копирование.
 * @throws `bad_alloc` при нехватке памяти для копирования.
 */
template<typename T>
TMatrix<T>::TMatrix(TMatrix&& other) noexcept : data(move(other.data)), rows(other.rows), cols(other.cols) {
    other.rows = 0;
    other.cols = 0;
}

/**
 * @brief Деструктор.
 */
template<typename T>
TMatrix<T>::~TMatrix() {}

/**
 * @brief Оператор присваивания.
 * Присваивает значение полей одной матрицы другой.
 *
 * @param other Правый операнд присваивания.
 * @return Ссылку на текущий объект (*this).
 */
template<typename T>
TMatrix<T>& TMatrix<T>::operator=(const TMatrix& other) {
    if (this != &other) {
        rows = other.rows;
        cols = other.cols;
        data = other.data;
    }
    return *this;
}
```

```
193     /**
194      * @brief Оператор перемещающего присваивания.
195      * Присваивает значение полей одной матрицы другой.
196      * При этом первая матрица очищается.
197      *
198      * @param other Правый операнд присваивания.
199      * @return Ссылку на текущий объект (*this).
200      */
201     template<typename T>
202     TMatrix<T>& TMatrix<T>::operator=(TMatrix&& other) noexcept {
203         if (this != &other) {
204             data = move(other.data);
205             rows = other.rows;
206             cols = other.cols;
207
208             other.rows = 0;
209             other.cols = 0;
210         }
211         return *this;
212     }
213
214     /**
215      * @brief Оператор сравнения.
216      * Проверяет, равны ли две матрицы (по размеру и содержимому).
217      *
218      * @param other Матрица для сравнения.
219      * @return true, если матрицы равны, иначе false.
220      */
221     template<typename T>
222     bool TMatrix<T>::operator==(const TMatrix& other) const {
223         return (rows == other.rows && cols == other.cols && data == other.data);
224     }
225
226     /**
227      * @brief Метод ввода матрицы из потока.
228      * Сначала запрашивает количество строк и столбцов из потока.
229      * Затем поэлементно запрашивает значения из потока.
230      * @param is Входной поток.
231      * @throws invalid_argument при вводе отрицательных размеров.
232      * @throws runtime_error при ошибках потока.
233      */
234     template<typename T>
235     void TMatrix<T>::input(istream& is) {
236         is >> rows;
237         if (is.fail()) {
238             throw runtime_error("TMatrix Input: Failed to read number of rows from stream.");
239         }
240         if (rows < 0) {
241             throw invalid_argument("TMatrix Input: Number of rows cannot be negative!");
242         }
243         is >> cols;
244         if (is.fail()) {
245             throw runtime_error("TMatrix Input: Failed to read number of columns from stream.");
246         }
247         if (cols < 0) {
248             throw invalid_argument("TMatrix Input: Number of columns cannot be negative!");
249         }
250
251         try {
252             data.resize(rows, vector<T>(cols));
253         }
254         catch (const bad_alloc& e) {
255             throw;
256         }
257
258         for (int i = 0; i < rows; ++i) {
259             for (int j = 0; j < cols; ++j) {
260                 is >> data[i][j];
261                 if (is.fail()) {
262                     throw runtime_error("TMatrix Input: Failed to read element [" + to_string(i) + "][" + to_string(j) + "] from stream.");
263                 }
```

```cpp
            }
        }
    }

    /**
     * @brief Метод вывода матрицы в консоль.
     * Выводит элементы по строкам и столбцам в виде матрицы.
     * @param os Выходной поток.
     */
    template<typename T>
    void TMatrix<T>::output(ostream& os) const {
        for (int i = 0; i < rows; ++i) {
            for (int j = 0; j < cols; ++j) {
                os << data[i][j] << " ";
                if (os.fail()) {
                    throw runtime_error("TMatrix Output: Failed to write element [" + to_string(i) + "][" + to_string(j) + "] to stream.");
                }
            }
            os << endl;
            if (os.fail()) {
                throw runtime_error("TMatrix Output: Failed to write newline after row " + to_string(i) + " to stream.");
            }
        }
    }

    /**
     * @brief Подсчёт элементов > 0 и < среднего по строке.
     * Для каждой строки вычисляется среднее арифметическое.
     * Затем подсчитывается количество элементов в строке, которые удовлетворяют условиям.
     *
     * @return Общее количество таких элементов во всей матрице.
     */
    template<typename T>
    int TMatrix<T>::countPositiveBelowRowMean() const {
        int count = 0; // Количество нужных элементов
        for (int i = 0; i < rows; ++i) {
            T sum = T(0); // Сумма элементов строки
            for (int j = 0; j < cols; ++j) {
                sum = sum + data[i][j];
            }
            T mean = sum / static_cast<T>(cols); // Среднее арифметическое по строке

            for (int j = 0; j < cols; ++j) {
                if (data[i][j] > T(0) && data[i][j] < mean) {
                    count++;
                }
            }
        }
        return count;
    }

    /**
     * @brief Расчёт суммы отрицательных элементов.
     * Перебирает все элементы матрицы и суммирует только те, что < T(0).
     *
     * @return Сумму отрицательных элементов нужного типа данных.
     */
    template<typename T>
    T TMatrix<T>::sumOfNegativeElements() const {
        T sum = T(0); // Сумма подходящих элементов
        /// Проходимся по каждому элементу и прибавляем значение к сумме, если меньше нуля
        for (int i = 0; i < rows; ++i) {
            for (int j = 0; j < cols; ++j) {
                if (data[i][j] < T(0)) {
                    sum = sum + data[i][j];
                }
            }
```

```
330             }
331         }
332         return sum;
333     }
334         PlayerUnknown14, 3 недели назад via  PR #28 • laba: лабы по ЯП (#28) …
335     /**
336      * @brief Метод генерации случайных значений.
337      * Заполняет матрицу случайными значениями в диапазоне `[min_val, max_val]`.
338      *
339      * @param min_val Минимальное значение для генерации.
340      * @param max_val Максимальное значение для генерации.
341      * @throws `invalid_argument` если `min_val > max_val`.
342      */
343     template<typename T>
344     void TMatrix<T>::GenerateRandomValues(T min_val, T max_val) {
345         if (min_val > max_val) {
346             throw invalid_argument("TMatrix GenerateRandomValues: min_val cannot be greater than max_val!");
347         }
348         // Используем статическую переменную для инициализации генератора один раз
349         static random_device rd;
350         static mt19937 gen(rd());
351         static uniform_real_distribution<double> dis(0.0, 1.0); // Генерим от 0 до 1
352
353         for (int i = 0; i < rows; ++i) {
354             for (int j = 0; j < cols; ++j) {
355                 double t = dis(gen);
356                 T range = max_val - min_val;
357                 T val = min_val + static_cast<T>(t) * range;
358                 data[i][j] = val;
359             }
360         }
361     }
362
363     /**
364      * @brief Модификация элемента матрицы вручную.
365      * Запрашивает новое значение для элемента по указанным индексам.
366      *
367      * @param row Индекс строки.
368      * @param col Индекс столбца.
369      * @param is Входной поток.
370      * @throws out_of_range если индексы выходят за границы.
371      * @throws runtime_error при ошибках потока.
372      */
373     template<typename T>
374     void TMatrix<T>::modifyElement(int row, int col, istream& is) {
375         try { // Проверяем границы матрицы
376             data.at(row).at(col);
377             cout << "Enter new value for [" << row << "][" << col << "]: ";
378             is >> data[row][col];
379         }
380         catch (const out_of_range&) {
381             throw;
382         }
383     }
384
385     /**
386      * @brief Установка случайного значения для одного элемента.
387      * Аналогично GenerateRandomValues, но только для одного элемента.
388      *
389      * @param row Индекс строки.
390      * @param col Индекс столбца.
391      * @param min_val Минимальное значение.
392      * @param max_val Максимальное значение.
393      * @throws out_of_range если индексы выходят за границы.
394      * @throws invalid_argument если min_val > max_val.
395      */
396     template<typename T>
397     void TMatrix<T>::SetRandomValue(int row, int col, T min_val, T max_val) {
```

```cpp
void TMatrix<T>::SetRandomValue(int row, int col, T min_val, T max_val) {
    if (min_val > max_val) {
        throw invalid_argument("TMatrix SetRandomValue: min_val cannot be greater than max_val!");
    }
    try { // Проверяем границы матрицы
        data.at(row).at(col);
    }
    catch (const out_of_range&) {
        throw;
    }

    // Используем статическую переменную для инициализации генератора один раз
    static random_device rd;
    static mt19937 gen(rd());
    static uniform_real_distribution<double> dis(0.0, 1.0); // Генерим от 0 до 1

    double t = dis(gen);
    T range = max_val - min_val;
    T val = min_val + static_cast<T>(t) * range;
    data[row][col] = val;
}

/**
 * @brief Загрузка матрицы из файла.
 * Формат файла: первые два числа — количество строк и столбцов.
 * Затем идут элементы по строкам.
 *
 * @param filename Имя файла для загрузки.
 * @throws runtime_error при ошибках файла.
 */
template<typename T>
void TMatrix<T>::loadFromFile(const string& filename) {
    ifstream file(filename);
    if (!file.is_open()) {
        throw runtime_error("TMatrix loadFromFile: Cannot open file " + filename);
    }

    file >> rows >> cols;
    data.resize(rows, vector<T>(cols));

    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            file >> data[i][j];
        }
    }
    file.close();
    cout << "Data loaded from " << filename << endl;
}

/**
 * @brief Сохранение матрицы в файл.
 * Формат файла: первые два числа — количество строк и столбцов.
 * Затем идут элементы по строкам.
 *
 * @param filename Имя файла для сохранения.
 * @throws runtime_error при ошибках файла.
 */
template<typename T>
void TMatrix<T>::saveToFile(const string& filename) const {
    ofstream file(filename);
    if (!file.is_open()) {
        throw runtime_error("TMatrix saveToFile: Cannot open file " + filename);
    }

    file << rows << " " << cols << endl;
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            file << data[i][j] << " ";
        }
        file << endl;
    }
    file.close();
    cout << "Data saved to " << filename << endl;
}

template<typename T>
const T& TMatrix<T>::at(int row, int col) const {
    return data.at(row).at(col);
}

#endif
```

Файл **Fraction.h**:

```
#ifndef FRACTION_H        PlayerUnknown14, 3 недели назад via  PR #28 • laba: лабы по ЯП (
#define FRACTION_H

#include <iostream>
#include <cmath>
#include <stdexcept>
#include <string>
#include <sstream>
using namespace std;

/// @brief Класс исключения для дробей, унаследован от `std::exception`
You, 6 часов назад | 1 author (You)
class FractionException : public exception {
private:
    string msg;
public:
    explicit FractionException(const string& message) : msg(message) {}
    const char* what() const noexcept override {
        return msg.c_str();
    }
};

/**
 * @brief Дробное число, состоящее из числителя и знаменателя.
 */
You, 6 часов назад | 2 authors (PlayerUnknown14 and one other)
class Fraction {
private:
    // Числитель
    int num;
    // Знаменатель
    int den;

    /**
     * @brief Вспомогательная функция для вычисления наибольшего общего делителя (НОД).
     * Используется для сокращения дроби.
     *
     * @param a Первое число.
     * @param b Второе число.
     * @return Наибольший общий делитель чисел a и b.
     */
    int gcd(int a, int b) const {
        a = abs(a); // Берем абсолютное значение, чтобы НОД был положительным
        b = abs(b);
        while (b) { // Алгоритм Евклида
            int temp = b;
            b = a % b;
            a = temp;
        }
        return a;
    }

    /**
     * @brief Вспомогательная функция для нормализации дроби.
     * Приводит дробь к каноническому виду: знаменатель положительный, дробь сокращена.
     */
    void normalize() {
        if (den < 0) { // Если знаменатель отрицательный, переносим знак на числитель
            num = -num;
            den = -den;
        }
        int g = gcd(num, den); // Вычисляем НОД
        if (g != 0) { // Если НОД не равен 0, сокращаем дробь
            num /= g;
            den /= g;
        }
    }

public:
```

```
67      public:
68          /**
69           * @brief Конструктор из двух целых чисел.
70           * Создаёт дробь num/den. Выполняет нормализацию.
71           *
72           * @param n Числитель. По умолчанию 0.
73           * @param d Знаменатель. По умолчанию 1.
74           * @throws FractionException если d == 0.
75           */
76          Fraction(int n = 0, int d = 1) : num(n), den(d) {
77              if (den == 0) {
78                  throw FractionException("Fraction Constructor: Denominator cannot be zero!");
79              }
80              normalize(); // Приводим к каноническому виду
81          }
82
83          /**
84           * @brief Конструктор из double.
85           * Преобразует вещественное число в дробь с ограниченной точностью.
86           * Например, 0.1 -> 1/10.
87           *
88           * @param d Вещественное число для преобразования.
89           */
90          Fraction(double d) {
91              const int precision = 100; // Максимальный знаменатель ~100 (2 знака после запятой)
92              double scaled_val = d * precision; // Масштабируем до целого
93              int n = static_cast<int>(round(scaled_val)); // Округляем до ближайшего целого
94              int g = gcd(n, precision); // НОД для сокращения
95              num = n / g;
96              den = precision / g;
97              normalize(); // Финальная нормализация
98          }
99
100         // Возвращает числитель
101         int getNum() const { return num; }
102         // Возвращает знаменатель
103         int getDen() const { return den; }
104
105         /**
106          * @brief Оператор "меньше".
107          * Сравнивает две дроби, приводя к общему знаменателю.
108          *
109          * @param other Вторая дробь для сравнения.
110          * @return true, если текущая дробь меньше, иначе false.
111          */
112         bool operator<(const Fraction& other) const {
113             return (num * other.den) < (other.num * den); // Приведение к общему знаменателю перед сравнением
114         }
115
116         /**
117          * @brief Оператор "больше".
118          *
119          * @param other Вторая дробь для сравнения.
120          * @return true, если текущая дробь больше, иначе false.
121          */
122         bool operator>(const Fraction& other) const {
123             return (num * other.den) > (other.num * den);
124         }
125
126         /**
127          * @brief Оператор "равно".
128          *
129          * @param other Вторая дробь для сравнения.
130          * @return true, если дроби равны (с учётом сокращения), иначе false.
131          */
132         bool operator==(const Fraction& other) const {
133             return (num == other.num && den == other.den);
134         }
135
136         /**
```

```cpp
136     /**      PlayerUnknown14, 3 недели назад via  PR #28 • laba: лабы по ЯП (#2
137      * @brief Оператор "не равно".
138      *
139      * @param other Вторая дробь для сравнения.
140      * @return true, если дроби не равны, иначе false.
141      */
142     bool operator!=(const Fraction& other) const {
143         return !(*this == other); // Использует operator==
144     }
145
146     /**
147      * @brief Оператор "меньше или равно".
148      *
149      * @param other Вторая дробь для сравнения.
150      * @return true, если текущая дробь <= other, иначе false.
151      */
152     bool operator<=(const Fraction& other) const {
153         return (*this < other) || (*this == other);
154     }
155
156     /**
157      * @brief Оператор "больше или равно".
158      *
159      * @param other Вторая дробь для сравнения.
160      * @return true, если текущая дробь >= other, иначе false.
161      */
162     bool operator>=(const Fraction& other) const {
163         return (*this > other) || (*this == other);
164     }
165
166     // Операторы арифметики
167     /**
168      * @brief Оператор сложения.
169      * Складывает две дроби: a/b + c/d = (a*d + c*b) / (b*d).
170      *
171      * @param other Вторая дробь.
172      * @return Новую дробь, равную сумме.
173      */
174     Fraction operator+(const Fraction& other) const {
175         return Fraction(num * other.den + other.num * den, den * other.den);
176     }
177
178     /**
179      * @brief Оператор вычитания.
180      * Вычитает: a/b - c/d = (a*d - c*b) / (b*d).
181      *
182      * @param other Вторая дробь.
183      * @return Новую дробь, равную разности.
184      */
185     Fraction operator-(const Fraction& other) const {
186         return Fraction(num * other.den - other.num * den, den * other.den);
187     }
188
189     /**
190      * @brief Оператор умножения.
191      * Умножает: a/b * c/d = (a*c) / (b*d).
192      *
193      * @param other Вторая дробь.
194      * @return Новую дробь, равную произведению.
195      */
196     Fraction operator*(const Fraction& other) const {
197         return Fraction(num * other.num, den * other.den);
198     }
199
200     /**
201      * @brief Оператор деления.
202      * Делит: `a/b / c/d = (a*d) / (b*c)`.
203      * Проверяет деление на ноль.
204      *
205      * @param other Дробь-делитель.
```

```
205     * @param other Дробь-делитель.        PlayerUnknown14, 3 недели назад via  PR #28 • laba: лабы по ЯП (#28) …
206     * @return Новую дробь, равную частному.
207     * @throws `FractionException` если `other.num == 0`.
208     */
209    Fraction operator/(const Fraction& other) const {
210        if (other.num == 0) {
211            throw FractionException("Fraction Division: Division by zero!");
212        }
213        return Fraction(num * other.den, den * other.num);
214    }
215
216    // Проверяет, положительна ли дробь
217    bool isPositive() const { return num > 0; }
218    // Проверяет, отрицательна ли дробь
219    bool isNegative() const { return num < 0; }
220
221    /**
222     * @brief Преобразование дроби в double.
223     * Может использоваться для приближённых вычислений или генерации случайных значений.
224     *
225     * @return Значение дроби как double.
226     */
227    double toDouble() const {
228        return static_cast<double>(num) / den;
229    }
230
231    /**
232     * @brief Оператор ввода из потока.
233     * Читает дробь в формате "num/den" или "num" (если знаменатель 1).
234     *
235     * @param is Входной поток (например, `cin` или `ifstream`).
236     * @param f Объект дроби для заполнения.
237     * @return Ссылку на входной поток.
238     * @throws `invalid_argument`, `out_of_range`, `FractionException`
239     */
240    friend istream& operator>>(istream& is, Fraction& f) {
241        string s;
242        is >> s; // Читаем всю строку до пробела
243
244        size_t pos = s.find('/'); // Ищем символ '/'
245        if (pos == string::npos) { // Если '/' нет — это целое число
246            try { // stoi (строка в целое) может выбросить invalid_argument или out_of_range
247                f.num = stoi(s);
248            }
249            catch (const invalid_argument& e) {
250                throw invalid_argument("Fraction Input: Cannot convert string to integer for numerator - " + s);
251            }
252            catch (const out_of_range& e) {
253                throw out_of_range("Fraction Input: Integer out of range for numerator - " + s);
254            }
255            f.den = 1;
256        } else { // Если '/' есть — разбиваем строку
257            string num_str = s.substr(0, pos); // До '/'
258            string den_str = s.substr(pos + 1); // После '/'
259            try {
260                f.num = stoi(num_str);
261            }
262            catch (const invalid_argument& e) {
263                throw invalid_argument("Fraction Input: Cannot convert string to integer for numerator - " + num_str);
264            }
265            catch (const out_of_range& e) {
266                throw out_of_range("Fraction Input: Integer out of range for numerator - " + num_str);
267            }
268            try {
269                f.den = stoi(den_str);
270            }
271            catch (const invalid_argument& e) {
272                throw invalid_argument("Fraction Input: Cannot convert string to integer for denominator - " + den_str);
273            }
```

```
273                 }        You, 6 часов назад • Uncommitted changes
274                 catch (const out_of_range& e) {
275                     throw out_of_range("Fraction Input: Integer out of range for denominator - " + den_str);
276                 }
277                 if (f.den == 0) {
278                     throw FractionException("Fraction Input: Denominator cannot be zero!");
279                 }
280             }
281             f.normalize(); // Нормализуем дробь после ввода
282             return is;
283         }
284
285         /**
286          * @brief Оператор вывода в поток.
287          * Выводит дробь в формате "num/den", если знаменатель != 1, иначе только "num".
288          *
289          * @param os Выходной поток (например, cout или ofstream).
290          * @param f Объект дроби для вывода.
291          * @return Ссылку на выходной поток.
292          */
293         friend ostream& operator<<(ostream& os, const Fraction& f) {
294             os << f.num; // Выводим числитель
295             if (f.den != 1) os << "/" << f.den; // Если знаменатель != 1, выводим "/"
296             return os;
297         }
298     };
299
300     #endif
```

Файл **main.cpp**:

```
#include "TMatrix.h"
#include "Fraction.h"
#include <iostream>
#include <fstream>
using namespace std;
        PlayerUnknown14, 3 недели назад via  PR #28 • laba: лабы по ЯП (#28) …
/// @brief Шаблонная функция, реализующая меню для конкретного типа данных
/// @tparam T тип данных элементов матрицы
/// @param datatype_name название типа данных
template<typename T>
void runMatrixMenu(string datatype_name) {
    TMatrix<T> matrix;
    bool running = true;

    while (running) {
        cout << "\n====================================" << endl;
        cout << "     TMatrix MENU (Type: " << datatype_name << ")" << endl;
        cout << "====================================" << endl;
        cout << "\n--- OPERATIONS ON MATRIX ---" << endl;
        cout << "1. Create new matrix from input" << endl;
        cout << "2. Display current matrix" << endl;
        cout << "3. Count positive elements below row mean" << endl;
        cout << "4. Sum of negative elements" << endl;
        cout << "5. Load matrix from file" << endl;
        cout << "6. Save current matrix to file" << endl;
        cout << "7. Modify an element (console input)" << endl;
        cout << "8. Set random value for an element" << endl;
        cout << "9. Generate random values for the whole matrix" << endl;
        cout << "10. Demonstration of copy constructor" << endl;
        cout << "11. Demonstration of move constructor" << endl;
        cout << "0. Go back to type selection" << endl;
        cout << "Choose an action: ";

        int choice;
        cin >> choice;

        try {
            switch (choice) {
                case 1: { // Ввод матрицы
                    matrix.input();
                    cout << "Matrix created." << endl;
                    break;
                }
                case 2: { // Вывод матрицы
                    cout << "\n--- Current Matrix ---" << endl;
                    cout << matrix << endl;
                    break;
                }
                case 3: { // Счёт положительных элементов > среднего значения
                    cout << "\nCount of positive elements below row mean: " << matrix.countPositiveBelowRowMean() << endl;
                    break;
                }
                case 4: { // Сумма отрицательных элементов
                    auto sum = matrix.sumOfNegativeElements();
                    cout << "\nSum of negative elements: " << sum << endl;
                    break;
                }
                case 5: { // Загрузка матрицы из файла
                    string filename;
                    cout << "Enter filename to load from: ";
                    cin >> filename;
                    matrix.loadFromFile(filename);
                    break;
                }
                case 6: { // Выгрузка матрицы в файл
                    string filename;
                    cout << "Enter filename to save to: ";
                    cin >> filename;
                    matrix.saveToFile(filename);
                    break;
                }
                case 7: { // Модификация элемента
```

```
72                  case 7: { // Модификация элемента
73                      int r, c;
74                      cout << "Enter row index: ";
75                      cin >> r;
76                      cout << "Enter column index: ";
77                      cin >> c;
78                      matrix.modifyElement(r, c, cin);
79                      break;
80                  }
81                  case 8: { // Задать случайное значение элементу
82                      int r, c;
83                      cout << "Enter row index: ";
84                      cin >> r;
85                      cout << "Enter column index: ";
86                      cin >> c;
87                      T min_val, max_val;
88                      cout << "Enter min value for random: ";
89                      cin >> min_val;
90                      cout << "Enter max value for random: ";
91                      cin >> max_val;
92                      matrix.SetRandomValue(r, c, min_val, max_val);
93                      break;
94                  }
95                  case 9: { // Сгенерировать случайные значения всем элементам матрицы
96                      T min_val, max_val;
97                      cout << "Enter min value for random: ";
98                      cin >> min_val;
99                      cout << "Enter max value for random: ";
100                     cin >> max_val;
101                     matrix.GenerateRandomValues(min_val, max_val);
102                     cout << "Random values generated." << endl;
103                     break;
104                 }
105                 case 10: { // Демонстрация конструктора копирования
106                     cout << "--- Demonstration of copy constructor ---" << endl;
107                     cout << "Current Matrix" << endl;
108                     cout << matrix << endl;
109                     TMatrix<T> temp_copy = matrix;
110                     cout << "Copy Of Current Matrix" << endl;
111                     cout << temp_copy << endl;
112                     break;
113                 }
114                 case 11: { // Демонстрация конструктора перемещения
115                     cout << "--- Demonstration of move constructor ---" << endl;
116                     cout << "Current Matrix" << endl;
117                     cout << matrix << endl;
118                     TMatrix<T> temp_moved = move(matrix);
119                     cout << "Matrix after move (should be empty)" << endl;
120                     cout << matrix << endl;
121                     cout << "Moved Matrix" << endl;
122                     cout << temp_moved << endl;
123                     // Восстанавливаем оригинальную матрицу для дальнейшей работы
124                     matrix = move(temp_moved);
125                     cout << "Original matrix restored." << endl;
126                     break;
127                 }
128                 case 0: { // Возврат к выбору типа
129                     running = false;
130                     cout << "Returning to type selection..." << endl;
131                     break;
132                 }
133                 default: { // Неверный ввод
134                     cout << "Invalid choice. Please try again." << endl;
135                 }
136             }
137         }
138         catch (const FractionException& e) {
139             cout << "Fraction Error: " << e.what() << endl;
140         }
```

```
140             }       You, 15 минут назад • Рабочая версия, I guess …
141             catch (const invalid_argument& e) {
142                 cout << "Invalid Argument Error: " << e.what() << endl;
143             }
144             catch (const runtime_error& e) {
145                 cout << "Runtime Error: " << e.what() << endl;
146             }
147             catch (const out_of_range& e) {
148                 cout << "Out of Range Error: " << e.what() << endl;
149             }
150             catch (const length_error& e) {
151                 cout << "Length Error (overflow): " << e.what() << endl;
152             }
153             catch (const bad_alloc& e) {
154                 cout << "Memory Allocation Error: " << e.what() << endl;
155             }
156             catch (...) {
157                 cout << "An unknown error occurred." << endl;
158             }
159         }
160     }
161
162     int main() {
163         try {
164             bool running = true;
165
166             while (running) {
167                 cout << "\n===================================" << endl;
168                 cout << "         MAIN MENU" << endl;
169                 cout << "===================================" << endl;
170                 cout << "\n--- SELECT MATRIX TYPE ---" << endl;
171                 cout << "1. Work with Float Matrix" << endl;
172                 cout << "2. Work with Fraction Matrix" << endl;
173                 cout << "0. Exit" << endl;
174                 cout << "Choose an action: ";
175
176                 int choice;
177                 cin >> choice;
178
179                 switch (choice) {
180                     case 1: { // Переключение на матрицу вещественных чисел
181                         cout << "Switched to Float Matrix." << endl;
182                         runMatrixMenu<float>("Float"); // Вызываем шаблонную функцию с типом float
183                         break;
184                     }
185                     case 2: { // Переключение на матрицу дробей
186                         cout << "Switched to Fraction Matrix." << endl;
187                         runMatrixMenu<Fraction>("Fraction"); // Вызываем шаблонную функцию с типом Fraction
188                         break;
189                     }
190                     case 0: { // Выход из программы
191                         running = false;
192                         cout << "Exiting..." << endl;
193                         break;
194                     }
195                     default: { // Неверный ввод
196                         cout << "Invalid choice. Please try again." << endl;
197                     }
198                 }
199             }
200         }
201         catch (const exception& e) {
202             // Перехватываем любые стандартные исключения, вышедшие из основного цикла (т. е. критические)
203             cout << "A critical error occurred in the main application: " << e.what() << endl;
204             cout << "Application will be terminated." << endl;
205             return 1; // Возвращаем код ошибки
206         }
207         catch (...) {
208             // Перехватываем любые другие исключения (т. е. критические)
209             cout << "An unknown critical error occurred in the main application." << endl;
```

```
207     catch (...) {
208         // Перехватываем любые другие исключения (т. е. критические)
209         cout << "An unknown critical error occurred in the main application." << endl;
210         cout << "Application will be terminated." << endl;
211         return 1; // Возвращаем код ошибки
212     }
213
214     return 0;
215 }
```

Создание матрицы с помощью ручного ввода и её вывод в консоль:

```
=====================================
         MAIN MENU
=====================================

--- SELECT MATRIX TYPE ---
1. Work with Float Matrix
2. Work with Fraction Matrix
0. Exit
Choose an action: 1
Switched to Float Matrix.

=====================================
     TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
11. Demonstration of move constructor
0. Go back to type selection
Choose an action: 1
Enter number of rows: 2
Enter number of columns: 2
Enter element [0][0]: 1
Enter element [0][1]: 4.0
Enter element [1][0]: -2.3
Enter element [1][1]: -16.7
Matrix created.
```

```
--- Current Matrix ---
1 4
-2.3 -16.7
```

Счёт положительных элементов ниже среднего арифметического значения строки:

```
=====================================
     TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
11. Demonstration of move constructor
0. Go back to type selection
Choose an action: 3

Count of positive elements below row mean: 1
```

Сумма отрицательных элементов матрицы:

```
=====================================
     TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
11. Demonstration of move constructor
0. Go back to type selection
Choose an action: 4

Sum of negative elements: -19
```

Изменение элемента матрицы (ручной ввод):

```
=====================================
     TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
11. Demonstration of move constructor
0. Go back to type selection
Choose an action: 7
Enter row index: 0
Enter column index: 1
Enter new value for [0][1]: 20.5
```

```
--- Current Matrix ---
1 20.5
-2.3 -16.7
```

Изменение элемента матрицы (рандомизация):

```
=====================================
     TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
11. Demonstration of move constructor
0. Go back to type selection
Choose an action: 8
Enter row index: 0
Enter column index: 0
Enter min value for random: 1
Enter max value for random: 5
```

```
--- Current Matrix ---
4.32172 20.5
-2.3 -16.7
```

Рандомизация всех элементов матрицы:

```
=====================================
     TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
11. Demonstration of move constructor
0. Go back to type selection
Choose an action: 9
Enter min value for random: 5.0
Enter max value for random: 10.0
Random values generated.
```

```
--- Current Matrix ---
8.81355 9.99781
9.817 8.24662
```

Копирование матрицы с помощью конструктора копирования:

```
=====================================
     TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
11. Demonstration of move constructor
0. Go back to type selection
Choose an action: 10
--- Demonstration of copy constructor ---
Current Matrix
8.81355 9.99781
9.817 8.24662

Copy Of Current Matrix
8.81355 9.99781
9.817 8.24662
```

Перемещение матрицы с помощью конструктора перемещения:

```
=====================================
     TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
11. Demonstration of move constructor
0. Go back to type selection
Choose an action: 11
--- Demonstration of move constructor ---
Current Matrix
8.81355 9.99781
9.817 8.24662

Matrix after move (should be empty)

Moved Matrix
8.81355 9.99781
9.817 8.24662

Original matrix restored.
```

```
--- Current Matrix ---
8.81355 9.99781
9.817 8.24662
```

Запись данных массива в файл:

```
=====================================
     TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
11. Demonstration of move constructor
0. Go back to type selection
Choose an action: 6
Enter filename to save to: text1.txt
Data saved to text1.txt
```

```
text1.txt M ×
C_codespace > 2 kurs > 3-rd senester > program
You, 28 секунд назад | 1 author (You)
1  2 2
2  8.81355 9.99781
3  9.817 8.24662
4
```

Переключение на матрицу с дробными числами:

```
=====================================
     TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
11. Demonstration of move constructor
0. Go back to type selection
Choose an action: 0
Returning to type selection...

=====================================
         MAIN MENU
=====================================

--- SELECT MATRIX TYPE ---
1. Work with Float Matrix
2. Work with Fraction Matrix
0. Exit
Choose an action: 2
Switched to Fraction Matrix.
```

Загрузка данных из файла:

```
=====================================
     TMatrix MENU (Type: Fraction)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
11. Demonstration of move constructor
0. Go back to type selection
Choose an action: 5
Enter filename to load from: input_fraction
Data loaded from input_fraction
```

```
--- Current Matrix ---
1/2 1/3
-1/4 -1/5
```

Счёт положительных элементов ниже среднего арифметического значения строки и сумма отрицательных элементов матрицы:

```
Count of positive elements below row mean: 1

Sum of negative elements: -9/20
```

Изменения элемента матрицы при вводе с консоли и рандомизация элемента матрицы:

```
=====================================
     TMatrix MENU (Type: Fraction)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
11. Demonstration of move constructor
0. Go back to type selection
Choose an action: 7
Enter row index: 0
Enter column index: 0
Enter new value for [0][0]: 2/10
```

```
=====================================
     TMatrix MENU (Type: Fraction)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
11. Demonstration of move constructor
0. Go back to type selection
Choose an action: 8
Enter row index: 0
Enter column index: 1
Enter min value for random: -1/2
Enter max value for random: 1/2
```

```
--- Current Matrix ---
1/5 -37/100
-1/4 -1/5
```

Рандомизация всех элементов матрицы:

```
=====================================
     TMatrix MENU (Type: Fraction)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
11. Demonstration of move constructor
0. Go back to type selection
Choose an action: 9
Enter min value for random: 1/20
Enter max value for random: 1/5
Random values generated.
```

```
--- Current Matrix ---
137/1000 337/2000
253/2000 16/125
```

Демонстрация работы различных исключений:

```
=====================================
     TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
11. Demonstration of move constructor
0. Go back to type selection
Choose an action: 1
Enter number of rows: -2
Invalid Argument Error: TMatrix Input: Number of rows cannot be negative!
```

```
=====================================
     TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
11. Demonstration of move constructor
0. Go back to type selection
Choose an action: 9
Enter min value for random: 100
Enter max value for random: 50
Invalid Argument Error: TMatrix GenerateRandomValues: min_val cannot be greater than max_val!
```

```
=====================================
     TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
11. Demonstration of move constructor
0. Go back to type selection
Choose an action: 5
Enter filename to load from: wrong_file.txt
Runtime Error: TMatrix loadFromFile: Cannot open file wrong_file.txt
```

```
=====================================
     TMatrix MENU (Type: Fraction)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
11. Demonstration of move constructor
0. Go back to type selection
Choose an action: 1
Enter number of rows: 2
Enter number of columns: 2
Enter element [0][0]: 1/0
Fraction Error: Fraction Input: Denominator cannot be zero!
```

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. **Какой принцип ООП реализуется с помощью шаблонных функций и классов?**
   Обобщение (Generics) — возможность создания кода, работающего с разными типами данных, без дублирования.
2. **Когда целесообразно использовать шаблоны функций?**
   Когда нужно выполнить одинаковую логику над данными разных типов.
3. **Каков общий вид объявления шаблона функции?**
   template<typename T> тип_возврата имя_функции(параметры)
4. **Каков синтаксис объявления шаблонного класса?**
   template<typename T> class ИмяКласса { ... };
5. **В чём особенности определения шаблонного класса?**
   Все методы и операторы внутри класса также становятся шаблонами.
6. **В чём особенности определения метода шаблонного класса вне класса?**
   Перед определением метода нужно снова указать template<typename T> (или template<class T>). Сигнатура метода включает имя шаблонного класса с параметром:
   template<typename T> void TClassName<T>::methodName(...) { ... }.
7. **Что такое специализация шаблона?**
   Это частичная или полная замена параметра шаблона конкретным типом, чтобы предоставить особую реализацию для этого типа.
8. **Как объявить объект шаблонного класса?**
   ИмяКласса<конкретный_тип> имя_объекта; (например, TMatrix<int> intMatrix;).
9. **Что такое инстанцирование?**
   Процесс создания конкретной версии шаблона (функции или класса) для определенного типа компилятором.
10. **Что такое исключительная ситуация?**
    Событие, возникающее во время выполнения программы, которое нарушает нормальный поток управления (например, деление на ноль, выход за границы массива, нехватка памяти). Такие ситуации обрабатываются с помощью механизма исключений.
11. **Для чего предназначены блоки try, catch, throw?**
    - try — оборачивает код, в котором может возникнуть исключение;
    - catch — перехватывает и обрабатывает выброшенное исключение;
    - throw — генерирует (выбрасывает) исключение.
12. **Как работает раскрутка стека?**
    Когда выбрасывается исключение, программа последовательно «раскручивает» стек вызовов: уничтожает локальные объекты и возвращается к предыдущим функциям, пока не найдёт подходящий catch-блок.

13. В какой последовательности программа обрабатывает исключительную ситуацию?

1. Выполняется throw — создаётся объект исключения.
2. Программа ищет ближайший try-блок в стеке вызовов.
3. Среди catch-блоков после этого try ищется совместимый по типу.
4. Блок найден — выполняется соответствующий catch.
5. Блок не найден — продолжается раскрутка стека выше.
6. Если вообще не найдено (исключение не перехвачено) — вызывается std::terminate().

14. Что произойдет, если исключение окажется не перехваченным?

Если исключение не будет перехвачено ни одним catch-блоком, программа вызовет функцию std::terminate(), что приведёт к аварийному завершению.

15. Как создать собственный класс исключения?

оздать класс, унаследованный от std::exception (или одного из его потомков, например std::runtime_error), и переопределить виртуальную функцию what(). Пример кастомного класса:

```
/// @brief Класс исключения для дробей, унаследован от `std::exception`
You, 7 часов назад | 1 author (You)
class FractionException : public exception {
private:
    string msg;
public:
    explicit FractionException(const string& message) : msg(message) {}
    const char* what() const noexcept override {
        return msg.c_str();
    }
};
```

16. В чем преимущество использования исключений перед возвратом кодов ошибок?

- Исключения не требуют постоянной проверки возвращаемого значения после каждого вызова.
- Позволяет отделить логику ошибок от основной логики программы.
- Обеспечивают безопасную передачу ошибок через несколько уровней вложенности.
- Позволяют передавать объекты с детальной информацией об ошибке, а не просто коды.

17. Чем оператор throw отличается от оператора return?

- **return** передаёт управление назад вызывающей в заданном месте программы.
- **throw** немедленно прерывает выполнение текущей функции и ищет обработчик исключения в коде программы, даже через несколько уровней вызова.
- После **throw** управление не возвращается к месту вызова (в отличие от **return**).

18. Что такое перехват исключений по ссылке и почему это важно?

Перехват по ссылке — это запись вида catch (const MyException& e). Это важно, т.к.:

- избегает срезки объекта (object slicing), если исключение — наследник.
- предотвращает лишнее копирование.
- позволяет корректно работать с полиморфными исключениями.

19. В каком месте среди блоков catch должен располагаться блок catch(…)?

Блок **catch**(...) должен быть последним, так как обработка исключений идёт сверху вниз, из-за чего при другом порядке расположения специфичные обработчики никогда не будут достигнуты.

20. Что означает идентификатор b в следующей строке с catch-блоком? catch (int b)

Имя переменной, в которую копируется значение, переданное в **throw**. Так, при throw 42; в блоке catch (int b) переменная b будет равна 42.

ВЫВОДЫ

Я получил практические навыки разработки и отладки программ с использованием шаблонов классов и механизмов обработки исключений в языке C++.